Real Estate Prediction

Réalisé par :
Mathis Aulagnier AULM12040200
Ryan Collobert COLR28120200
Moïse Tsomanya KUMM06100000
Cours : Fondamentaux de l'apprentissage automatique (8INF867)

1 Présentation du dataset

Le dataset intitulé *Ames Housing Dataset - Kaggle Real Estate Prediction* a été utilisé dans le cadre de la compétition Kaggle *House Prices - Advanced Regression Techniques*, lancée le 3 juillet 2017. Cette compétition vise à prédire le prix de vente des maisons situées à Ames, dans l'Iowa. Ce problème de prédiction immobilière s'inscrit dans la catégorie des problèmes de régression supervisée et d'analyse prédictive.

Caractéristiques du dataset : Le dataset contient un total de 2 930 observations. Chaque observation est caractérisée par 81 variables, dont 80 variables explicatives et une variable cible, `SalePrice`, qui représente le prix de vente des maisons en dollars US.

Le dataset n'est pas propre, certaines variables contiennent des valeurs manquantes, comme `LotFrontage` ou `GarageYrBlt`. Une étape de prétraitement est nécessaire. Cette dernière inclue l'encodage des variables catégorielles, ordinales et la gestion des valeurs manquantes.

Métrique utilisée : La métrique utilisée pour évaluer les performances des modèles dans cette compétition est l'erreur quadratique moyenne logarithmique (*Root Mean Squared Logarithmic Error*, RMSLE). Cette métrique est calculée à l'aide de la formule suivante :

$$RMSLE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\log(y_i+1) - \log(\hat{y}_i+1))^2},$$

où y_i représente les valeurs réelles des prix de vente, et $\hat{y}_i$ les valeurs prédites.

2 Analyse des données

Lors de nos premières tentatives de modélisation, les résultats obtenus étaient très insatisfaisants. Après une analyse approfondie, nous avons identifié que la principale faiblesse de nos modèles provenait d'une analyse de données inadéquate. Nous avons donc décidé de concentrer nos efforts sur un travail méthodique d'exploration et de prétraitement des données. La première étape a été de catégoriser nos données en deux types : numériques et catégoriques (ou objets).

2.1 Traitement des données numériques

Nous avons commencé par examiner nos données numériques en les représentant graphiquement, notamment en les visualisant par rapport à la variable cible, `SalePrice`. Cette approche nous a rapidement permis de détecter des anomalies. Ces anomalies ne correspondaient pas à des erreurs de collecte de données, mais à des maisons présentant des caractéristiques atypiques ou extrêmes, rendant leur intégration dans l'entraînement du modèle peu pertinente. Pour améliorer la qualité des données, nous avons éliminé ces observations atypiques comme illustré dans la figure ci-dessous.

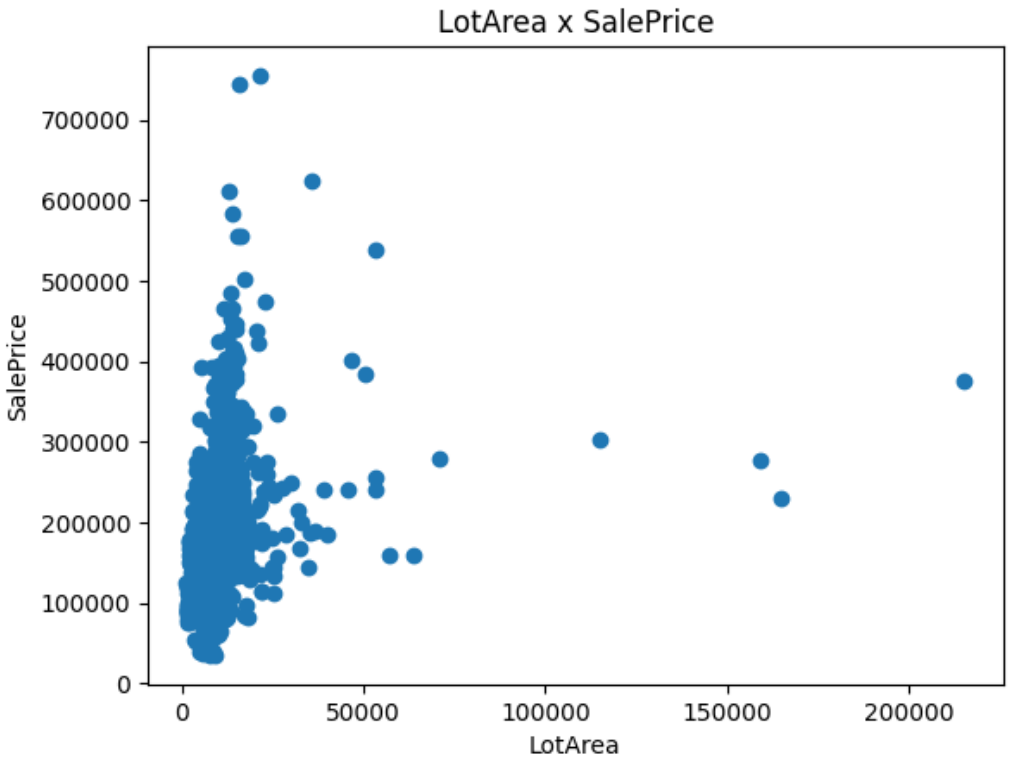


Figure 1: Visualisation des données et élimination des anomalies.

Cependant, cette méthode présentait une limite importante : elle ne permettait qu'une analyse en deux dimensions, alors que notre jeu de données repose sur un espace de 38 dimensions. Représenter ces données de manière visuelle et intuitive était donc impossible. Pour résoudre ce problème, nous avons eu recours à un autoencodeur.

2.1.1 Nettoyage des données grâce à un autoencodeur

Un autoencodeur est un modèle de deep learning constitué de deux parties : un encodeur et un décodeur. L'encodeur projette les données d'entrée dans un espace latent réduit, tandis que le décodeur tente de reconstruire les données originales à partir de cet espace latent. L'objectif est d'apprendre une fonction identité qui minimise la différence entre les données d'entrée et de sortie. En d'autres termes, le modèle cherche à satisfaire la condition suivante : $g \circ f(x) \approx x$, où f est la fonction de l'encodeur et g , celle du décodeur.

Pour nettoyer notre dataset, nous avons utilisé un autoencodeur et évalué sa capacité à reconstruire les données. Si l'autoencodeur échouait à reconstruire correctement certaines observations, cela indiquait que ces données étaient très différentes des autres et posaient problème. À l'aide de la métrique MSE (erreur quadratique moyenne de reconstruction), nous avons défini un seuil basé sur la moyenne (M) et l'écart type (σ). Les observations dont l'erreur dépassait $M + \sigma$ ont été supprimées.

Nous avons utilisé un autoencodeur basé sur un perceptron multicouche (MLP) pour réduire nos 38 dimensions à un espace latent de 16 dimensions. La figure suivante montre les observations éliminées grâce à cette méthode.

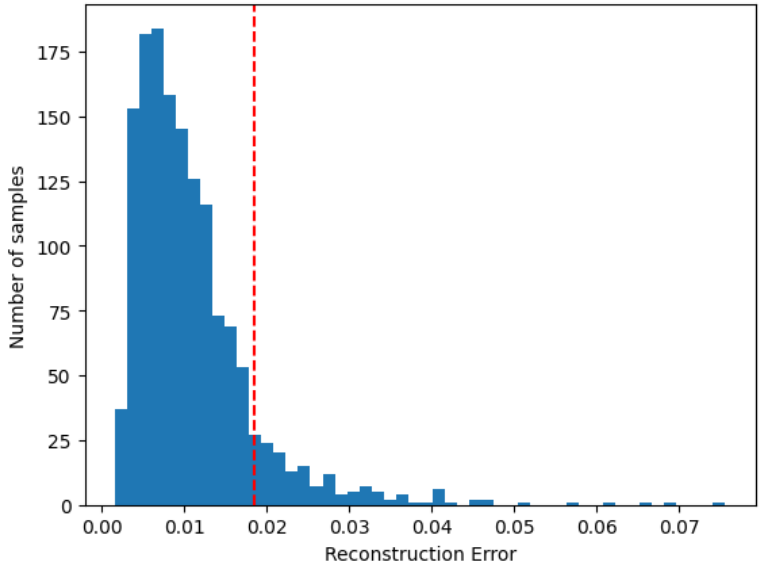


Figure 2: Histogramme des erreurs de reconstruction et données supprimées.

Au total, 160 observations numériques ont été supprimées, améliorant ainsi la qualité globale du dataset.

2.2 Traitement des données catégoriques

Le traitement des variables catégoriques s'est articulé autour de trois axes principaux : la gestion des valeurs manquantes, l'association et la corrélation.

2.2.1 Gestion des valeurs manquantes

Nous avons d'abord identifié les variables présentant des valeurs manquantes et calculé leur fréquence. Toutes les variables ayant plus de 1 000 valeurs manquantes ont été supprimées, car leur faible taux de remplissage rendait leur exploitation difficile. Parmi ces variables, on retrouve par exemple la taille de la piscine.

Pour les variables ayant moins de 1 000 valeurs manquantes, nous avons adopté une stratégie différente : remplacer les valeurs manquantes par `Unknown`. Dans certains cas spécifiques, nous avons cherché à associer une valeur probable. Par exemple, pour la variable liée au matériau de la façade, nous avons analysé un graphique en chandelier. Celui-ci montrait une forte ressemblance statistique (moyenne et écart type similaires) entre les maisons avec une façade inconnue et celles construites en briques communes (`BrkCmn`). Nous avons donc imputé les valeurs manquantes avec cette catégorie. La figure ci-dessous illustre ce raisonnement.

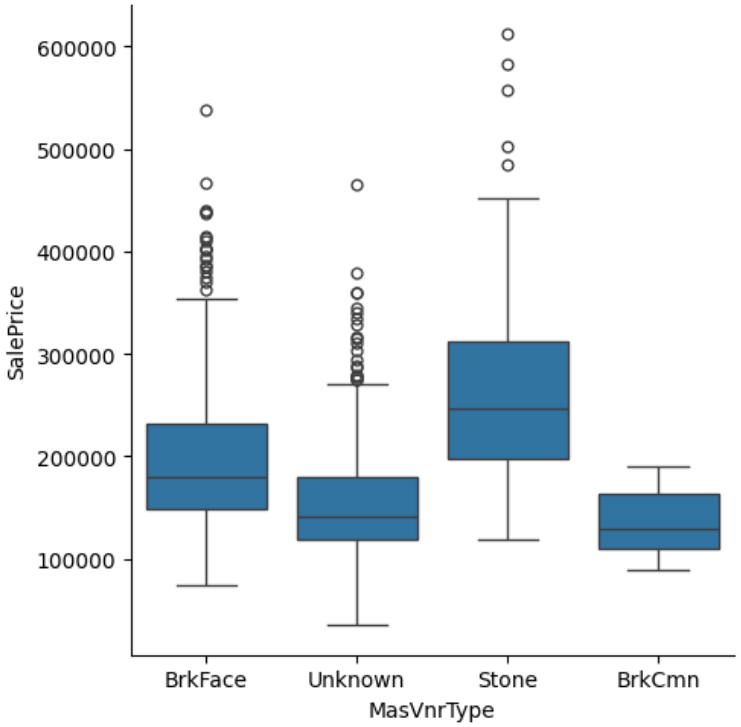


Figure 3: Graphique en chandelier pour l'association des valeurs manquantes.

2.2.2 Corrélation entre variables

Après avoir géré les valeurs manquantes, nous avons observé la matrice de corrélation des variables. Certaines variables numériques et catégoriques présentaient des corrélations très fortes. Par exemple, la capacité du garage à accueillir des voitures et la surface totale du garage étaient fortement corrélées.

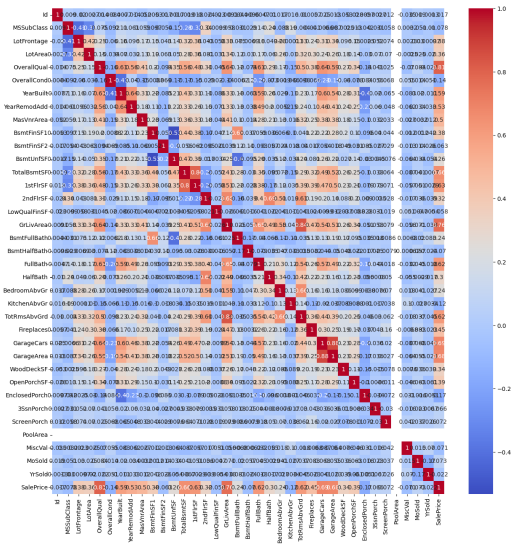


Figure 4: Exemple de corrélation entre deux variables liées au garage.

Ces corrélations posent problème, car elles peuvent entraîner une redondance d'information, voire biaiser le modèle en accordant un poids excessif à des variables similaires. Nous avons donc supprimé l'une des variables de chaque paire fortement corrélée afin de réduire ce risque.

2.3 Construction de la pipeline de prétraitement

Après avoir traité nos données, nous avons construit une pipeline robuste. Pour les données numériques, nous avons utilisé un imputer pour remplacer les valeurs manquantes par leur moyenne, suivi d'une normalisation. Quant aux variables catégoriques, nous avons identifié deux types :

- Les variables catégoriques classiques, pour lesquelles nous avons remplacé les valeurs manquantes par la modalité la plus fréquente, avant d'appliquer un encodage `one-hot`.
- Les variables ordinales, pour lesquelles nous avons utilisé un encodage ordinal après avoir imputé les valeurs manquantes.

Une fois cette pipeline finalisée, nous avons entamé la construction de nos modèles prédictifs.

3 Science des données

Dans le cadre de ce projet, nous avons opté pour l'utilisation de cinq méthodes de régression différentes. Ces méthodes ont été sélectionnées en fonction de leur pertinence pour des problèmes de régression linéaire simple et des relations non linéaires complexes.

3.1 Méthodes adaptées aux problèmes de régression linéaire simple

3.1.1 Linear Regression (Régression linéaire)

La régression linéaire est une méthode statistique et d'apprentissage automatique qui modélise la relation entre une ou plusieurs variables indépendantes (features) et une variable dépendante (cible ou target).

Cette méthode repose sur l'hypothèse que la relation entre les variables est linéaire, c'est-à-dire que la variable cible peut être exprimée comme une combinaison linéaire des features. Elle minimise la somme des carrés des écarts entre les valeurs prédites et les valeurs réelles, un processus mesuré par l'erreur quadratique moyenne (Mean Squared Error, MSE).

3.1.2 Ridge Regression

La Ridge regression est une variante régularisée de la régression linéaire. Elle applique une pénalité basée sur la régularisation L2, ce qui réduit l'amplitude des coefficients sans les annuler complètement. Contrairement à la régularisation L1, qui annule complètement les coefficients et est utilisée dans la méthode Lasso Regression, une autre variante de la régression linéaire.

Cette régularisation aide à diminuer le surapprentissage (overfitting) et améliore les performances du modèle dans les cas de multicolinéarité, où les features sont fortement corrélées entre elles.

3.2 Méthodes adaptées aux problèmes de régression avec des relations non linéaires complexes

3.2.1 Random Forest Regressor

Le Random Forest Regressor est une méthode d'ensemble qui génère plusieurs arbres de décision sur des sous-échantillons aléatoires du jeu de données. La prédiction finale est obtenue en calculant la moyenne des prédictions de tous les arbres, ce qui permet de réduire le surapprentissage et d'augmenter la robustesse du modèle.

3.2.2 Gradient Boosting Regressor

Le Gradient Boosting Regressor est une méthode d'ensemble qui construit des modèles séquentiels, souvent des arbres de décision peu profonds. Chaque modèle est entraîné pour corriger les erreurs résiduelles des modèles précédents en utilisant une approche de descente de gradient. Le résultat final est une combinaison pondérée des prédictions de tous les modèles.

3.2.3 XGBoost (Extreme Gradient Boosting)

XGBoost est une version optimisée de l'algorithme Gradient Boosting, intégrant des améliorations en termes de performances, telles que la gestion efficace des échantillons, la régularisation pour éviter le surajustement, et une gestion des données manquantes.

Contrairement à l'implémentation classique du Gradient Boosting, XGBoost tire parti de techniques de parallélisation, ce qui le rend extrêmement rapide. Lors de nos tests, l'entraînement d'un modèle XGBoost a pris environ 10 secondes, contre 5 minutes pour un modèle Gradient Boosting Regressor. En outre, XGBoost est l'un des algorithmes les plus utilisés lors des compétitions Kaggle.

3.3 Combinaison de modèles : Voting Regressor et Stacking Regressor

Après avoir entraîné ces cinq modèles de régression (`lr`, `rfr`, `xgb`, `ridge` et `gdr`), leurs performances respectives se sont révélées insuffisantes pour atteindre nos objectifs prédictifs. Pour améliorer ces résultats, nous avons combiné les prédictions de ces modèles en utilisant deux approches : le **Voting Regressor** et le **Stacking Regressor**.

3.3.1 Voting Regressor

Le *Voting Regressor* combine les prédictions de plusieurs modèles de régression en calculant une moyenne pondérée des prédictions de chaque modèle.

Nous avons défini un `Voting Regressor` avec les cinq modèles mentionnés ci-dessus. Une grille de recherche a été utilisée pour optimiser les poids associés à chaque modèle, en testant différentes distributions (`weights`) des contributions de chaque prédicteur. Le Voting Regressor a permis une amélioration notable des prédictions par rapport aux modèles individuels. Grâce à cette architecture, nous avons pu rentrer dans le *top1000* de la compétition Kaggle.

3.3.2 Stacking Regressor

Le *Stacking Regressor* est une méthode avancée d'ensemble qui consiste à entraîner un modèle final, appelé **méta-modèle**, sur les prédictions de plusieurs modèles de base (ou *base models*). Contrairement au Voting Regressor, où la combinaison est fixe (par moyenne ou moyenne pondérée), le méta-modèle apprend à pondérer et ajuster les prédictions en fonction des erreurs et des relations complexes entre les modèles.

Pour cette approche, nous avons utilisé une combinaison des mêmes modèles de base (`lr`, `rfr`, `xgb`, `ridge`, `gdr`) et ajouté le Voting Regressor comme prédicteur supplémentaire. Le méta-modèle choisi pour le Stacking Regressor était une **régression linéaire**, après avoir comparé ses performances à celles d'autres modèles comme XGBoost, Random Forest ou Ridge. La régression linéaire a démontré une capacité supérieure à exploiter les relations entre les prédictions des modèles de base, aboutissant à des performances meilleures

En termes de résultats, cette méthode nous a permis de nous positionner à la $554^{\text{ème}}$ place, avec une erreur (RMSLE) de 13 134.53 .

4 Interface web

4.1 Choix d'outils d'implémentation de l'interface graphique

Pour le développement de l'interface graphique, nous avons utilisé le Framework ReactJS, un outil populaire et performant pour la création d'applications web interactives. ReactJS permet une gestion efficace de l'état et offre une structure modulaire grâce à ses composants réutilisables. En complément, nous avons intégré des bibliothèques comme reactstrap pour le design des composants afin d'assurer une expérience utilisateur (UX) fluide et professionnelle.

4.2 Architecture de l'application

L'application est structurée selon une architecture client-serveur :

- **Front-end** : développé avec ReactJS pour fournir une interface utilisateur dynamique et intuitive.
- **Back-end** : implémenté avec un serveur Flask Python qui permet la communication entre le front-end et notre modèle entraîné.
- **Modèles d'IA** : intégrés au back-end, le modèle est chargé de prédire les prix en fonction des données d'entrée fournies par l'utilisateur.

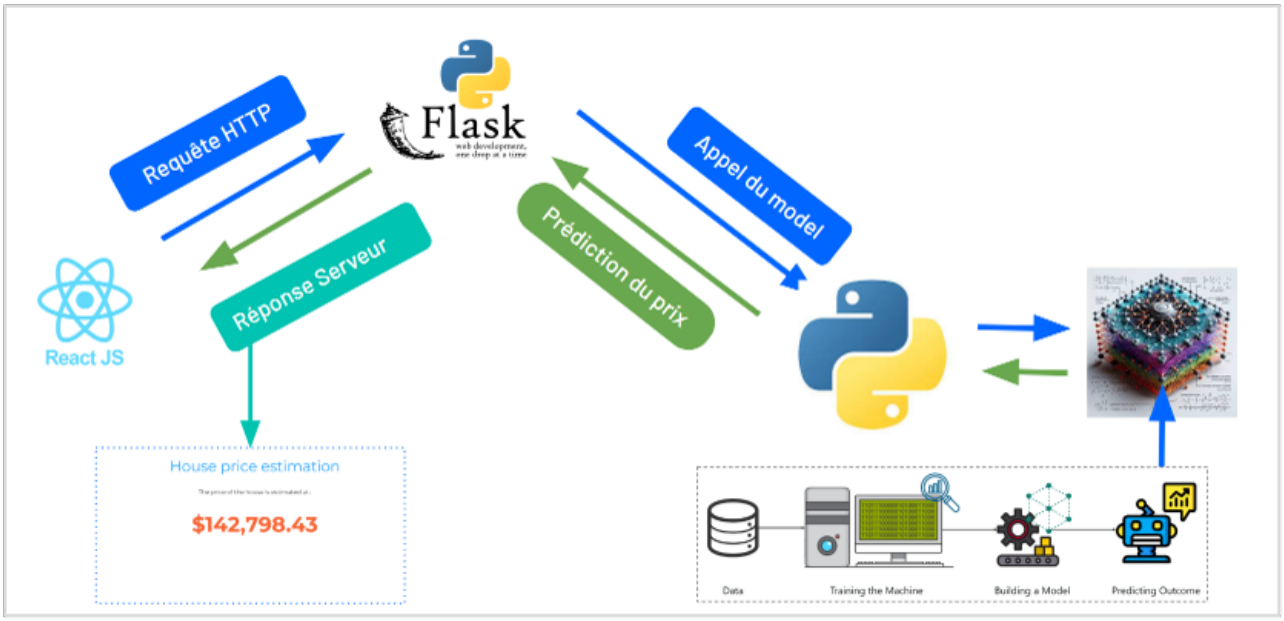


Figure 5: Architecture de l'application

4.3 Front-end

Le front-end est conçu pour :

- **Saisir les caractéristiques de la maison** : Permettre aux utilisateurs de saisir les caractéristiques d'une maison afin de prédire son prix. Les champs couvrent un large éventail de données (Informations générales, Caractéristiques du terrain, Localisation, Caractéristiques du bâtiment, Extérieur et fondation, Sous-sol, Intérieur, Équipements et finitions, Espaces extérieurs et autres détails...), que l'utilisateur peut rentrer, regroupées selon différents types (String, Integer, Boolean, Categorical).
- **Afficher les prédictions** : Les résultats de l'estimation du prix de la maison du modèle IA sont affichés en USD.
- **Expérience utilisateur** : Les animations et transitions fluides améliorent l'engagement, tandis que la conception responsive assure une compatibilité avec les appareils mobiles et de bureau.

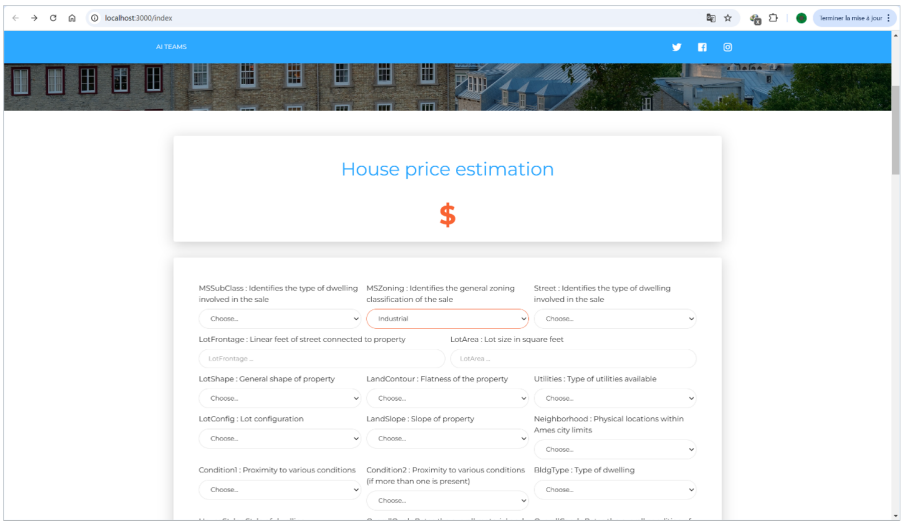


Figure 6: Interface de l'application (Questionnaires)

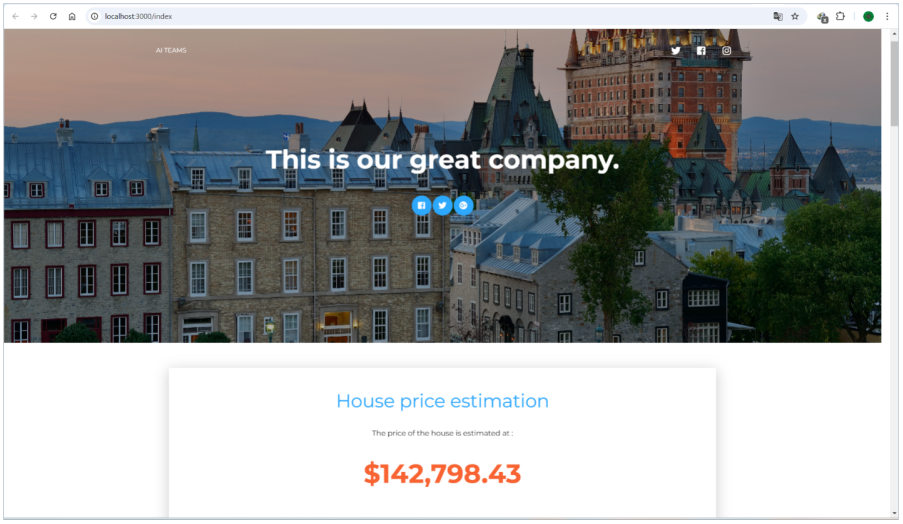


Figure 7: Interface de l'application (Affichage du prix)

5 Conclusion

Ce projet avait pour objectif de développer un modèle prédictif capable d'estimer avec précision les prix des maisons à Ames, Iowa. En combinant un travail approfondi d'analyse des données pour leur nettoyage avec des techniques avancées d'apprentissage automatique, nous avons réalisé des progrès significatifs vers cet objectif.

Grâce à un processus rigoureux de nettoyage des données, incluant l'élimination des valeurs aberrantes et la gestion des valeurs manquantes, nous avons renforcé la fiabilité des prédictions. L'utilisation d'un autoencodeur s'est révélée particulièrement efficace pour identifier et éliminer les points de données anormaux susceptibles de dégrader la performance du modèle.

L'exploration de divers algorithmes d'apprentissage automatique, notamment les techniques de boosting et de stacking, nous a permis d'obtenir des prédictions initiales prometteuses. De plus, la combinaison de ces modèles via des approches d'ensemble, telles que le vote et l'empilement, a considérablement amélioré la précision prédictive globale.

En complément, le développement d'une interface web intuitive renforce l'utilité pratique de notre modèle. Cette interface permet aux utilisateurs de saisir les caractéristiques d'une maison et de recevoir des estimations de prix précises, tout en offrant une expérience utilisateur fluide et accessible.

En conclusion, ce projet a démontré la puissance du Machine Learning dans l'analyse prédictive des prix de l'immobilier. Nous estimons avoir atteint notre objectif initial de développer un modèle précis et performant. Ce travail nous a permis de mieux comprendre l'importance de la qualité des données dans les projets d'apprentissage automatique, ainsi que l'efficacité des méthodes d'ensemble pour optimiser la précision des prédictions. Quant aux perspectives futures, nous pourrions ajouter l'intégration de données chronologiques afin d'analyser les tendances historiques des prix. De plus, nous pourrions essayer des algorithmes de Deep Learning comme l'expérimentation avec des réseaux de neurones. Enfin, nous pourrions essayer notre approche avec d'autres villes pour tester la généralisation de notre modèle.